1. Modifica la Práctica 01 para procesar las notas de 10 estudiantes: calcula el promedio, la nota máxima y mínima, cuántos aprobaron (nota ≥ 10.5) y el porcentaje de aprobados. Si menos del 60 % aprobó, muestra «Programar reforzamiento».

ej01_sin_escalar.py 4

Semana13_IA_Java > ej01_sin_escalar.py > ...

```python
# A) CON escalado
esc = StandardScaler()
km_con = KMeans(n_clusters=5, n_init=10, random_state=42).fit(esc.fit_transform(df))
c_con = pd.DataFrame(esc.inverse_transform(km_con.cluster_centers_), columns=df.columns).round(1)
print("\nCentroides CON StandardScaler:")
print(c_con.sort_values("ingreso_mil").to_string(index=False))

# B) SIN escalado
km_sin = KMeans(n_clusters=5, n_init=10, random_state=42).fit(df)
c_sin = pd.DataFrame(km_sin.cluster_centers_, columns=df.columns).round(1)
print("\nCentroides SIN StandardScaler:")
print(c_sin.sort_values("ingreso_mil").to_string(index=False))
print(f"Inercia sin escalar: {km_sin.inertia_:.2f}")

# C) Caso extremo: ingreso en SOLES
d = pd.DataFrame({
    "ingreso_soles": df["ingreso_mil"] * 1000,
    "puntaje_gasto": df["puntaje_gasto"]
})
km_s = KMeans(n_clusters=5, n_init=10, random_state=42).fit(d)
c_s = pd.DataFrame(km_s.cluster_centers_, columns=d.columns).round(1)
c_s["clientes"] = pd.Series(km_s.labels_).value_counts().sort_index().values
print("\nSIN escalar, ingreso en soles:")
print(c_s.sort_values("ingreso_soles").to_string(index=False))
```

Problems 4 Output Debug Console Terminal Ports Spell Checker 40 Python

```
Página de códigos activa: 65001

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>C:\Users\danie\AppData\Local\Programs\Python\Python312\python.exe "e:/SENATI - TODO/4° CICLO/4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312/PRACTICAS EN CLASE/Semana13_IA_Java/ej01_sin_escalar.py"
Desviación estándar de cada columna:
ingreso_mil      31.6
puntaje_gasto    28.4
dtype: float64

Centroides CON StandardScaler:
 ingreso_mil  puntaje_gasto
        24.1           19.6
        27.6           80.3
        70.1           51.2
        94.8           20.4
        95.5           84.4

Centroides SIN StandardScaler:
 ingreso_mil  puntaje_gasto
        24.1           19.6
        27.6           80.3
        70.1           51.2
        94.8           20.4
        95.5           84.4
Inercia sin escalar: 8440.29

SIN escalar, ingreso en soles:
 ingreso_soles  puntaje_gasto  clientes
       21963.3           41.3        39
       29545.7           58.2        41
       69724.4           52.2        39
       89335.9           48.6        25
       97529.0           53.4        56

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>
```

Este ejercicio demuestra cómo la programación orientada a datos permite automatizar la toma de decisiones institucionales. Al calcular métricas estadísticas básicas y contrastarlas con un umbral porcentual (60 %), el sistema deja de ser un simple calculador y pasa a generar una alerta operativa ("Programar reforzamiento") útil para la gestión académica

2. Crea un record Producto con los atributos código, precio, categoría, talla (S/M/L), enOferta y stock. Indica el tipo de dato de cada atributo y muestra: precio promedio, productos por categoría y productos con talla M o superior.

ej02_cambiar_k.py 5

Semana13_IA_Java > ej02_cambiar_k.py > ...

```python
# ej02_cambiar_k.py
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

script_dir = Path(__file__).resolve().parent
datos_dir = script_dir / "datos"
datos_dir.mkdir(exist_ok=True)
csv_path = datos_dir / "clientes.csv"

if not csv_path.exists():
    np.random.seed(42)
    c1 = np.random.normal(loc=[25, 20], scale=5, size=(40, 2))
    c2 = np.random.normal(loc=[28, 80], scale=5, size=(40, 2))
    c3 = np.random.normal(loc=[70, 50], scale=5, size=(40, 2))
    c4 = np.random.normal(loc=[95, 85], scale=5, size=(40, 2))
    c5 = np.random.normal(loc=[95, 20], scale=5, size=(40, 2))
    data = np.vstack([c1, c2, c3, c4, c5])
    df_temp = pd.DataFrame(data, columns=["ingreso_mil", "puntaje_gasto"]).clip(5, 100)
    df_temp.to_csv(csv_path, index=False)

df = pd.read_csv(csv_path)
esc = StandardScaler()
X = esc.fit_transform(df)

```

Problems 5 Output Debug Console Terminal Ports Spell Checker 17

```
E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>C:\Users\danie
\AppData\Local\Programs\Python\Python312\python.exe "e:/SENATI - TODO/4° CICLO/4 - MÓD PAQ MACHINE LEARN  PYTH
ON - PIAD 425 - 312/PRACTICAS EN CLASE/Semana13_IA_Java/ej02_cambiar_k.py"

=== k = 3 | inercia = 128.56 | silueta = 0.582 ===
 ingreso_mil  puntaje_gasto  clientes
        24.1           19.6        40
        27.6           80.3        40
        86.8           52.0       120

=== k = 5 | inercia = 9.59 | silueta = 0.810 ===
 ingreso_mil  puntaje_gasto  clientes
        24.1           19.6        40
        27.6           80.3        40
        70.1           51.2        40
        94.8           20.4        40
        95.5           84.4        40

=== k = 7 | inercia = 7.71 | silueta = 0.635 ===
 ingreso_mil  puntaje_gasto  clientes
        22.8           16.8        26
        26.7           24.9        14
        27.6           80.3        40
        69.8           54.5        26
        70.5           45.1        14
        94.8           20.4        40
        95.5           84.4        40

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>^B
```

Refuerza el principio fundamental de que cada tipo de dato exige operaciones específicas (por ejemplo, promediar precios numéricos frente a contar frecuencias en categorías nominales u ordenar en variables ordinales). El uso de record y enum en Java optimiza la estructuración de datasets limpios antes de pasarlos por un algoritmo de IA

3. En la Práctica 03 cambia la división a 70 % / 30 % y la semilla a 7. ¿Cómo cambian la ecuación, el MAE y el R^2? Explica por qué.

ej03_kmeans_k3.py 5

Semana13_IA_Java > ej03_kmeans_k3.py > ...

```python
    distancias = np.linalg.norm(puntos[:, None] - centroides[None, :], axis=2)
    grupo = distancias.argmin(axis=1)
    nuevos = np.array([puntos[grupo == j].mean(axis=0) for j in range(k)])
    print(f"Grupos: {grupo}")
    print(f"Nuevos centroides: {nuevos.round(2).tolist()}")

    if np.allclose(nuevos, centroides):
        print(f"Los centroides no cambiaron: el algoritmo CONVERGIÓ en la iteración {it}.")
        break
    centroides = nuevos

inercia = ((puntos - centroides[grupo]) ** 2).sum()
print(f"Inercia final: {inercia:.2f}")
```

Problems 5 Output Debug Console Terminal Ports Spell Checker 42

```
E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>C:\Users\danie
\AppData\Local\Programs\Python\Python312\python.exe "e:/SENATI - TODO/4° CICLO/4 - MÓD PAQ MACHINE LEARN  PYTH
ON - PIAD 425 - 312/PRACTICAS EN CLASE/Semana13_IA_Java/ej03_kmeans_k3.py"
Centroides iniciales iteración 1:
 [[1. 2.]
 [5. 8.]
 [8. 2.]]
Grupos: [0 0 0 1 1 1 2 2]
Nuevos centroides: [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
Centroides iniciales iteración 2:
 [[1.16666667 1.46666667]
 [7.33333333 9.        ]
 [8.5        2.5       ]]
Grupos: [0 0 0 1 1 1 2 2]
Nuevos centroides: [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
Los centroides no cambiaron: el algoritmo CONVERGIÓ en la iteración 2.
Inercia final: 16.98

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>^B
```

Evidencia la naturaleza estocástica del aprendizaje automático. Modificar la semilla aleatoria altera los subconjuntos de entrenamiento y prueba, lo que demuestra que el rendimiento de un modelo predictivo (medido en su MAE y R^2) depende críticamente de la calidad y representatividad de la partición de los datos

4. En la Práctica 04 clasifica la fruta (172 g, textura 5) con k = 1 y con k = 5. ¿Cambia la respuesta? ¿Por qué conviene que k sea impar?

ej04_notas.py 5

Semana13_IA_Java > ej04_notas.py > ...

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Crear datos ficticios (60 estudiantes, notas de 0 a 20)[cite: 31]
rng = np.random.default_rng(7)
perfiles = [(17, 16, 15), (15, 8, 15), (8, 15, 15), (9, 8, 15)] # (media Python, media Mate, n)[cite: 31]
filas = [np.column_stack([rng.normal(mp, 1.5, n), rng.normal(mm, 1.5, n)]) for mp, mm, n in perfiles]
X = np.vstack(filas).clip(0, 20).round(1)

df = pd.DataFrame(X, columns=["nota_python", "nota_matematica"])
df.insert(0, "estudiante", [f"E{i:02d}" for i in range(1, len(df) + 1)])
df.to_csv("notas.csv", index=False)
print("Filas generadas:", len(df))

# 2. Escalar y evaluar k con la silueta[cite: 31]
datos = df[["nota_python", "nota_matematica"]]
esc = StandardScaler()
Xe = esc.fit_transform(datos)

print(" k | silueta")
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xe)
    print(f"{k:>2} | {silhouette_score(Xe, km.labels_):.3f}")

# 3. Entrenar con el mejor k (k=4) e interpretar[cite: 31]
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(Xe)
df["grupo"] = km.labels_
c = pd.DataFrame(esc.inverse_transform(km.cluster_centers_), columns=datos.columns).round(1)
c["estudiantes"] = df["grupo"].value_counts().sort_index().values
print("\nCentroides y perfiles finales:")
print(c)
```

Problems 5 | Output | Debug Console | Terminal | Ports | Spell Checker 31 | Python

```
E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>C:\Users\danie\AppData\Local\Programs\Python\Python312\python.exe "e:/SENATI - TODO/4° CICLO/4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312/PRACTICAS EN CLASE/Semana13_IA_Java/ej04_notas.py"
Filas generadas: 60
 k | silueta
 2 | 0.446
 3 | 0.586
 4 | 0.640
 5 | 0.585
 6 | 0.535

Centroides y perfiles finales:
   nota_python  nota_matematica  estudiantes
0         14.6              8.4           16
1          7.7             15.1           15
2         16.8             14.9           15
3          8.5              7.8           14

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>
```

Ilustra la sensibilidad del algoritmo de los k vecinos más cercanos frente a los hiperparámetros. Un k pequeño (como 1) puede sobreajustarse a ruidos o valores atípicos, mientras que utilizar un número impar es una regla de diseño lógica indispensable para garantizar una mayoría absoluta y prevenir empates irresolubles en la votación

5. Completa la tabla indicando si cada problema es supervisado o no supervisado y si es clasificación, regresión o agrupamiento: a) predecir el precio de un departamento; b) detectar correos spam; c) formar grupos de clientes con gustos parecidos; d) predecir la temperatura de mañana; e) diagnosticar si una radiografía muestra neumonía; f) organizar noticias por temas sin categorías previas.

ej05_ic_niveles.py 4

Semana13_IA_Java > ej05_ic_niveles.py > ...

```python

if not csv_path.exists():
    np.random.seed(42)
    c1 = np.random.normal(loc=[25, 20], scale=5, size=(40, 2))
    c2 = np.random.normal(loc=[28, 80], scale=5, size=(40, 2))
    c3 = np.random.normal(loc=[70, 50], scale=5, size=(40, 2))
    c4 = np.random.normal(loc=[95, 85], scale=5, size=(40, 2))
    c5 = np.random.normal(loc=[95, 20], scale=5, size=(40, 2))
    data = np.vstack([c1, c2, c3, c4, c5])
    df_temp = pd.DataFrame(data, columns=["ingreso_mil", "puntaje_gasto"]).clip(5, 100)
    df_temp.to_csv(csv_path, index=False)

df = pd.read_csv(csv_path)
X = StandardScaler().fit_transform(df)
df["grupo"] = KMeans(n_clusters=5, n_init=10, random_state=42).fit_predict(X)

print(f"{'grupo':>5} {'nivel':>6} {'media':>7} {'lim_inf':>9} {'lim_sup':>9} {'ancho':>7} {'t':>7}")
for g, datos in df.groupby("grupo")["puntaje_gasto"]:
    n = len(datos)
    media = datos.mean()
    ee = stats.sem(datos)
    for nivel in (0.90, 0.95, 0.99):
        li, ls = stats.t.interval(nivel, df=n-1, loc=media, scale=ee)
        t = stats.t.ppf((1 + nivel) / 2, df=n-1)
        print(f"{g:5d} {nivel:6.0%} {media:7.2f} {li:9.2f} {ls:9.2f} {ls-li:7.2f} {t:7.3f}")
```

Problems 4 Output Debug Console Terminal Ports Spell Checker 24 Python

```
E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>C:\Users\danie\AppData\Local\Programs\Python\Python312\python.exe "e:/SENATI - TODO/4° CICLO/4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312/PRACTICAS EN CLASE/Semana13_IA_Java/ej05_ic_niveles.py"
grupo  nivel   media   lim_inf   lim_sup   ancho      t
    0    90%   80.32     79.13     81.51    2.38  1.685
    0    95%   80.32     78.89     81.75    2.86  2.023
    0    99%   80.32     78.41     82.24    3.83  2.708
    1    90%   20.43     19.22     21.64    2.43  1.685
    1    95%   20.43     18.97     21.89    2.92  2.023
    1    99%   20.43     18.48     22.38    3.90  2.708
    2    90%   19.63     18.23     21.03    2.80  1.685
    2    95%   19.63     17.95     21.31    3.36  2.023
    2    99%   19.63     17.38     21.88    4.50  2.708
    3    90%   84.38     83.35     85.40    2.05  1.685
    3    95%   84.38     83.15     85.61    2.46  2.023
    3    99%   84.38     82.73     86.02    3.29  2.708
    4    90%   51.21     49.67     52.75    3.08  1.685
    4    95%   51.21     49.36     53.06    3.70  2.023
    4    99%   51.21     48.74     53.69    4.95  2.708

E:\SENATI - TODO\4° CICLO\4 - MÓD PAQ MACHINE LEARN  PYTHON - PIAD 425 - 312\PRACTICAS EN CLASE>
```

Permite consolidar el criterio analítico para seleccionar el paradigma de Machine Learning adecuado según la disponibilidad de etiquetas históricas y el objetivo del negocio, diferenciando con claridad entre predecir un valor continuo, una clase discreta o descubrir patrones ocultos por agrupamiento